■■ Bank Management System — 7-Part Team Workload Breakdown

This document provides a balanced, modular division of the **Bank Management System** codebase into **7 equal parts**. Each module is self-contained with explicit file ownership, defined responsibilities, exact line numbers, API interfaces, and key deliverables for a team of 7 developers.

■ Module Assignments & Responsibility Breakdown

■ Member 1: Database Infrastructure & Auto-Provisioning

- **Primary Scope:** Core database connectivity, singleton config, auto-schema creation, and SQL DDL.
- **Owned Files & Line Ranges:**
  - **`db.DBConfig.java`** — Lines 1 to 241 (241 lines)
  - **`schema.sql`** — Lines 1 to 83 (83 lines)
- **Core Responsibilities:**
  1. Maintain thread-safe **`DBConfig`** Singleton using double-checked locking.
  2. Manage JDBC driver loading (**`com.mysql.cj.jdbc.Driver`**) and configurable connection properties (**`db.url`**, **`db.username`**, **`db.password`**).
  3. Maintain self-healing database auto-creation (**`createDatabaseAndSchema`**) when catching MySQL Error 1049 (Unknown database).
  4. Manage **`schema.sql`** DDL definitions, table constraints (**`chk_user_role`**, **`chk_account_type`**, **`chk_account_status`**, **`chk_trans_type`**), and default seed data.
  5. Provide resource cleanup helpers (**`closeConnection`**, **`closeResources`**).
- **Key Interfaces/Deliverables:**
  - **`DBConfig.getInstance().getConnection()`**
  - **`DBConfig.closeResources(ResultSet, Statement, Connection)`**
  - Executable **`schema.sql`** script.

■ Member 2: Domain Models & Data Transfer Objects

- **Primary Scope:** Core object-oriented entity design, data validation, and Java 17 DTO records.
- **Owned Files & Line Ranges:**
  - **`classes.User.java`** — Lines 1 to 61 (Abstract Base, 61 lines)
  - **`classes.Admin.java`** — Lines 1 to 33 (33 lines)
  - **`classes.Customer.java`** — Lines 1 to 83 (83 lines)
  - **`classes.Account.java`** — Lines 1 to 89 (89 lines)
  - **`classes.Transaction.java`** — Lines 1 to 92 (92 lines)
  - **`classes.CustomerAccountDTO.java`** — Lines 1 to 14 (14 lines)
- **Core Responsibilities:**
  1. Maintain class hierarchy (**`User`** abstract superclass extended by **`Admin`** and **`Customer`**).
  2. Implement financial models (**`Account`** with status/balance management and **`Transaction`** ledger entry).
  3. Maintain Java 17 **`record CustomerAccountDTO`** combining customer profile and account state for UI views.
  4. Ensure field encapsulation, data type safety (**`long`** account numbers, **`double`** currency), constructors, and string representations (**`toString()`**).
- **Key Interfaces/Deliverables:**
  - Encapsulated domain entities and DTO records used across DAO and UI layers.

■ Member 3: Authentication & Customer Onboarding DAO

- **Primary Scope:** Data Access Layer for security, authentication, and atomic customer onboarding.
- **Owned Files & Line Ranges:**
  - **`db.DatabaseOperations.java`** — Lines 1 to 191 (Auth & Onboarding Methods, 191 lines)
- **Core Responsibilities:**
  1. Implement **`validateLogin(username, password, role)`** using PreparedStatements and SQL Text Blocks to authenticate Admins and Customers.
  2. Implement multi-table atomic transaction **`createCustomerWithAccount(Customer, Account)`**:
     - Insert into **`users`** → generate **`user_id`**.
     - Insert into **`customers`** → generate **`customer_id`**.
     - Insert into **`accounts`** → create active account.
     - Insert opening deposit log into **`transactions`** if initial balance > 0.
  3. Manage explicit JDBC transaction demarcation (**`setAutoCommit(false)`**, **`commit()`**, **`rollback()`**) with suppressed exception safety.
- **Key Interfaces/Deliverables:**
  - **`User validateLogin(String username, String password, String role)`**
  - **`boolean createCustomerWithAccount(Customer customer, Account account)`**

■ Member 4: Financial Transactions & Audit Ledger DAO

- **Primary Scope:** Data Access Layer for financial operations, money movement, and ledger history.
- **Owned Files & Line Ranges:**
  - **`db.DatabaseOperations.java`** — Lines 192 to 613 (Financial Transaction & Query Methods, 422 lines)
- **Core Responsibilities:**
  1. Implement **`deposit(accountNumber, amount)`** with SQL balance updates and ledger entries.
  2. Implement **`withdraw(accountNumber, amount)`** with balance validation and atomic deduction.
  3. Implement **`transferFunds(sourceAccount, targetAccount, amount)`**:
     - Execute deterministic update ordering based on ascending account number (**`sourceAccount < targetAccount`**) to prevent MySQL deadlocks.
     - Write dual transaction audit logs for sender ("Fund Transfer Sent") and receiver ("Fund Transfer Received").
  4. Implement ledger queries: **`getAccountDetails`**, **`getAccountByCustomerId`**, **`getTransactionHistory`**, **`getAllCustomersWithAccounts`**, and **`getAllTransactions`**.
- **Key Interfaces/Deliverables:**
  - **`boolean deposit(long accountNumber, double amount)`**
  - **`boolean withdraw(long accountNumber, double amount)`**
  - **`boolean transferFunds(long sourceAccount, long targetAccount, double amount)`**
  - **`List getTransactionHistory(long accountNumber)`**
  - **`List getAllCustomersWithAccounts()`**

■ Member 5: Main Entry Point & Authentication UI

- **Primary Scope:** Application launcher, Swing Look & Feel configuration, and Login GUI.
- **Owned Files & Line Ranges:**
  - o **`App.java`** — Lines 1 to 26 (26 lines)
  - o **`ui.LoginFrame.java`** — Lines 1 to 183 (183 lines)
  - o **`run.bat`** — Lines 1 to 11 (11 lines)
- **Core Responsibilities:**
  1. Configure application entry point (**`App.main`**) on the Swing Event Dispatch Thread (**`SwingUtilities.invokeLater`**).
  2. Apply Look and Feel (Nimbus / System L&F;).
  3. Design **`LoginFrame`** Swing interface (**`GridBagLayout`** form with role selector, username, password, default button handling).
  4. Implement authentication controller connecting UI inputs to Member 3's **`validateLogin()`**.
  5. Route authenticated users to **`AdminDashboard`** or **`CustomerDashboard`**.
  6. Maintain runner script (**`run.bat`**) with proper **`-cp "bin;lib/*"`** classpath.
- **Key Interfaces/Deliverables:**
  - o Executable Swing login screen and launcher script.

■ Member 6: Customer Portal UI & Self-Service Banking

- **Primary Scope:** Customer dashboard UI, real-time balance tracking, and transaction operations interface.
- **Owned Files & Line Ranges:**
  - o **`ui.CustomerDashboard.java`** — Lines 1 to 345 (345 lines)
- **Core Responsibilities:**
  1. Build Customer Dashboard layout (header, account summary banner card, transaction table).
  2. Implement dynamic state management (**`refreshAccountState`**) to instantly update available balance and account status (**`accountInfoLabel`**).
  3. Create action dialogs and handlers for:
     - ■ Cash Deposit (**`handleDeposit`**)
     - ■ Cash Withdrawal (**`handleWithdraw`**)
     - ■ Fund Transfer (**`handleTransfer`** with positive account validation)
  4. Render real-time **`JTable`** transaction history with custom formatting (**`$%,.2f`**).
- **Key Interfaces/Deliverables:**
  - o Interactive Customer Swing Dashboard frame.

■ Member 7: Admin Control Panel & System Audit UI

- **Primary Scope:** Administrative dashboard, customer onboarding UI, directory management, and system audit logs.
- **Owned Files & Line Ranges:**
  - o **`ui.AdminDashboard.java`** — Lines 1 to 414 (414 lines)
- **Core Responsibilities:**
  1. Design Admin Control Panel using **`JTabbedPane`** (Registration, Directory, Audit Log).
  2. Implement customer registration form (**`createCustomerRegistrationPanel`**) with strict validation (phone number length, email format, non-negative deposits, input parsing).
  3. Implement thread-safe account number auto-generator (**`ThreadLocalRandom`**).
  4. Implement Customer Directory table with **`"N/A"`** fallback handling for accounts.
  5. Implement System Audit Ledger table displaying all bank-wide transactions.
  6. Implement automatic table re-loading via **`JTabbedPane`** tab change listeners.
- **Key Interfaces/Deliverables:**
  - o Interactive Admin Swing Dashboard frame.

■ Workload Balance Matrix

Member	Domain / Layer	Primary Files & Line Ranges	Workload Focus	Complexity
Member 1	DB Infrastructure	DBConfig.java (L1-241) schema.sql (L1-83)	Connection Pool, Auto-DB Creation, SQL Schema	Standard
Member 2	Domain & DTOs	User, Admin, Customer, Account, Transaction, DTO (372 lines total)	Object Model, Encapsulation, Records	Standard
Member 3	Auth & Onboarding DAO	DatabaseOperations.java (L1-191)	SQL Statements, Generated Keys, Transactions	High
Member 4	Financial DAO	DatabaseOperations.java (L192-613)	Concurrency, Lock Ordering, Audit Ledgers	High
Member 5	Entry & Login UI	App.java (L1-26) LoginFrame.java (L1-183) run.bat (L1-11)	Swing L&F;, Auth Controller, Build Scripts	Standard
Member 6	Customer UI	CustomerDashboard.java (L1-345)	Banner Cards, Dialogs, Real-Time JTable	High
Member 7	Admin UI	AdminDashboard.java (L1-414)	Tabbed Panels, Form Validation, Audit Views	High

■ Integration & Contract Dependencies

1. **Member 1** → **Members 3 & 4:** Members 3 & 4 rely on Member 1's **`DBConfig.getInstance().getConnection()`** for database access.
2. **Member 2** → **Members 3, 4, 6, 7:** All DAO and UI members consume Member 2's domain classes (**`Customer`**, **`Account`**, **`Transaction`**, **`CustomerAccountDTO`**).
3. **Member 5** → **Members 3, 6, 7:** Member 5 authenticates via Member 3's **`validateLogin`** and instantiates Member 6 (**`CustomerDashboard`**) or Member 7 (**`AdminDashboard`**).
4. **Member 6 & 7** → **Member 4:** Dashboards invoke Member 4's financial methods (**`deposit`**, **`withdraw`**, **`transferFunds`**, **`getTransactionHistory`**, **`getAllTransactions`**).